Memory Management

Lecture# 3

Introduction

- Memory management is the effective allocation of memory to processes on a computer system
- The principal operation of memory management is to bring processes into main memory for execution by the processor
- Multi-Programming
  - Multiple processes should be able to execute concurrently
  - Memory management has to allocate memory among processes
- Modern operating systems
  - Virtual memory management, based on paging and segmentation

Introduction

- An operating system has five principal storage management responsibilities
  - Process isolation
  - Automated allocation and management
  - Support for modular programming
  - Protection and access control
  - Long-term storage

- Goals
  - Support enough active processes to keep CPU busy
  - Use memory efficiently (minimize waste of memory)
  - Keep memory management overhead small

Requirements

- Relocation
  - Arbitrary positioning of code and data in physical memory
  - Virtual memory management
- Protection
  - Separation between processes in memory, needs hardware support
- Sharing
  - Allow shared access to memory by different processes
- Logical Organization
  - User programs have a modular structure, each process is divided into segments (code, data, stack), mechanisms for protecting segments (read-only, execute-only) and sharing among processes
- Physical Organization
  - Addressing of physical memory
  - Virtual memory based on a two-level hierarchy between memory and disk space
  - CPU can only access data in registers and memory

Introduction

- Program must be brought into memory and placed within a process for it to be run.
- To improve both the utilization of the CPU and the speed of its response to users, a general-purpose computer must keep several processes in memory.
- OS provides Many memory-management schemes to improve memory utilization.
- Input queue or job queue – collection of processes on the disk that are waiting to be brought into memory to run the program.
- OS has to perform various steps in order to execute the processes such as address and data binding.

Addresses Binding

- There are two types of addresses: logical address and physical address.
- Logical address is generated by CPU during a program execution. It is also called as virtual address.
- Physical Address identifies a physical location in a memory unit.
- Set of all logical addresses generated by CPU in reference to a program is referred as Logical Address Space.
- Set of all physical addresses mapped to the corresponding logical addresses is referred as Physical Address Space.
- Address binding is the process of mapping from one address space to another address space.

Addresses Binding

- Address Binding can happen at three different stages:

1. Compile-time Address Binding:

- It will be done before loading the program into memory.
- Compiler is responsible for performing address binding.
- The compiler requires interacts with an OS memory manager to perform compile-time address binding.

2. Load time Address Binding:

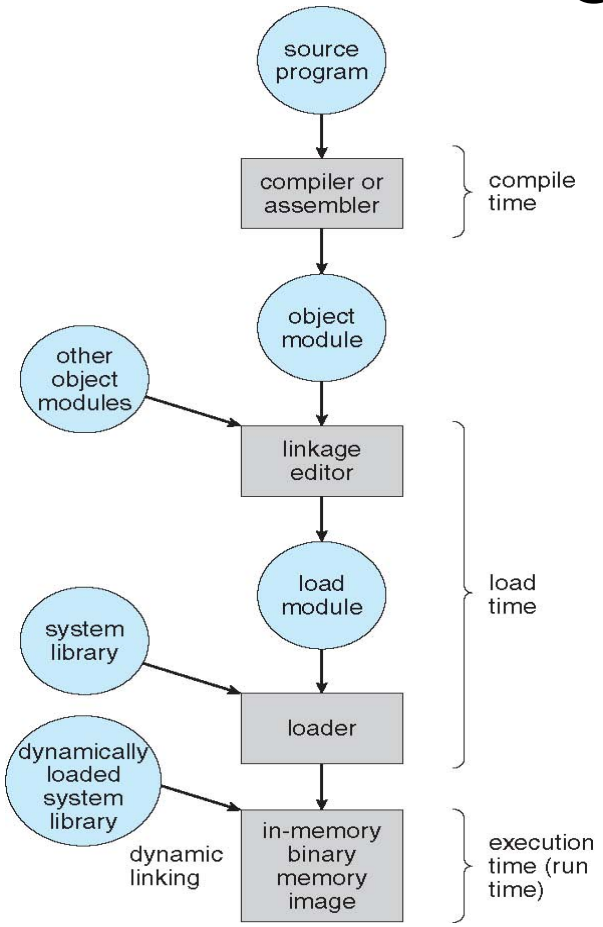
- It will be done after loading the program into memory.
- This type of address binding will be done by the OS memory manager i.e loader.

Addresses Binding

1. Execution time / Dynamic Address Binding:

- Binding done by the processor at the time of program execution.
- Need hardware support for address maps.
- The program will be kept on changing the locations in memory until the time of program execution.

Addresses Binding



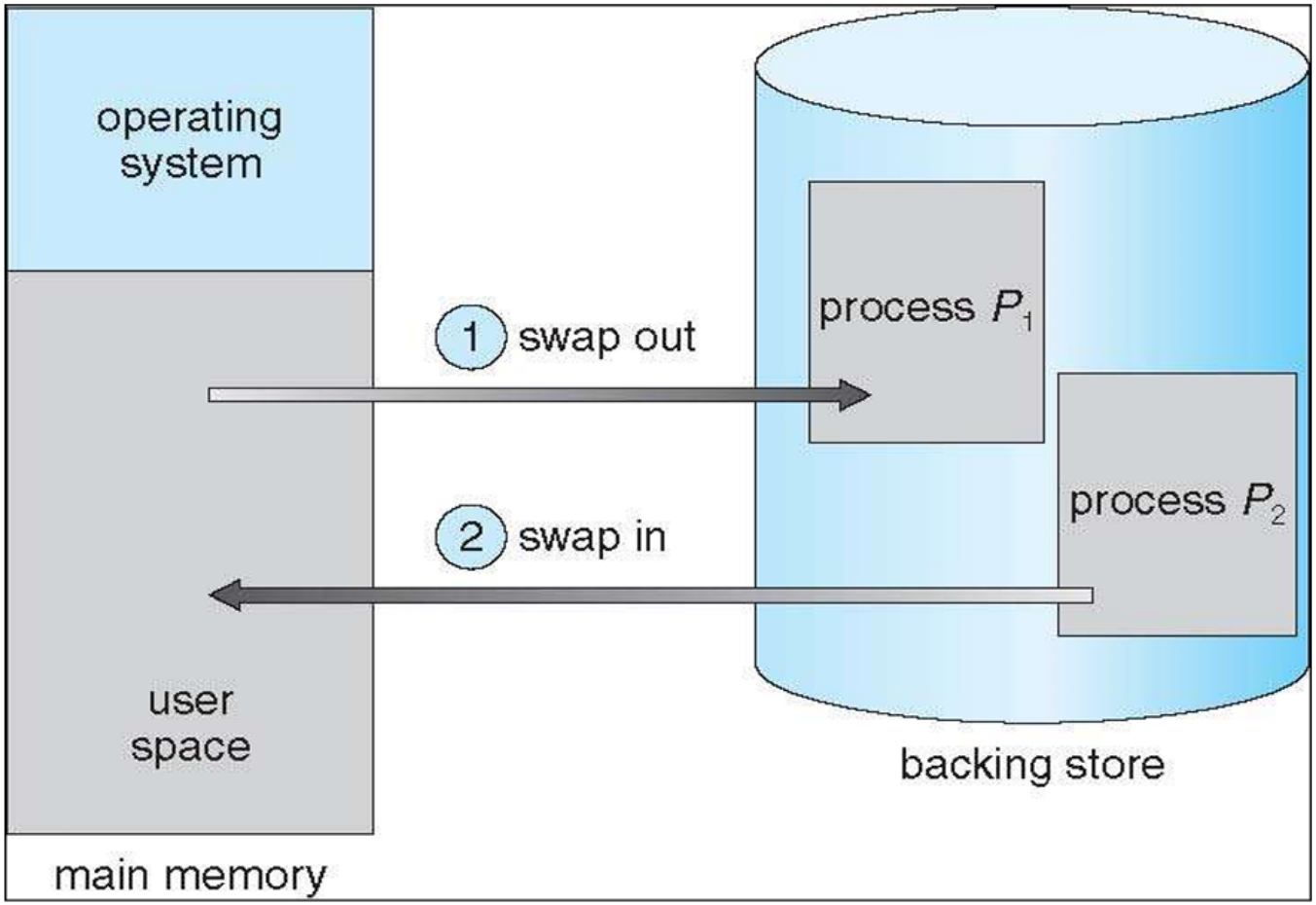
MMU(Memory Management Unit)

- The run time mapping between Virtual address and Physical Address is done by hardware device known as MMU.
- In memory management, Operating System will handle the processes and moves the processes between disk and memory for execution.
- It keeps the track of available and used memory.

Swapping

- Swapping is a technique in which a process can be moved temporarily out of memory to Backing Store (HD or SSD), and then brought back into memory for continued execution.
- ***Backing store***– is a fast disk large enough to accommodate copies of all memory images for all processes; and must provide direct access to these memory images.
- The swapping procedures that are found on modern operating systems (i.e., UNIX, Linux, and Windows) are:
  - Swapping is normally disabled
  - Swapping will be started if memory demand is more than certain threshold amount of memory.
  - Swapping will be disabled again once memory demand is reduced below certain threshold.

Schematic View of Swapping



Memory Allocation Methods

- Generally, there are two methods used for partitioning the memory allocation.
  - Contiguous memory allocation
  - Non-Contiguous memory allocation

Contiguous Memory Allocation

- The main memory must accommodate both the operating system and the various user processes.
- Therefore we need to allocate main memory in the most efficient way possible.
- The memory is usually divided into two partitions: one for the resident operating system and one for the user processes.
- In contiguous memory allocation, each process is contained in a single section of memory that is contiguous to the section containing the next process.
- Simplest methods for allocating memory is to divide memory into several fixed-sized partitions.
- Each partition may contain exactly one process. Thus, the degree of multiprogramming depends on number of partitions.

Contiguous memory management schemes:

- ➢ Single Partition
  - In this scheme, the main memory is divided into two contiguous areas or partitions. The operating systems reside permanently in one partition, generally at the lower memory, and the user process is loaded into the other partition.
- ➢ Multiple Partition
  - The operating system divides the available main memory into multiple parts to load multiple processes into the main memory.

- Fixed Partitioning: Each partition or block is of fixed size and can hold single process.
- Dynamic Partitioning: In this scheme the partitions used are of variable size. Each process occupies only as much memory as they require when loaded for processing.

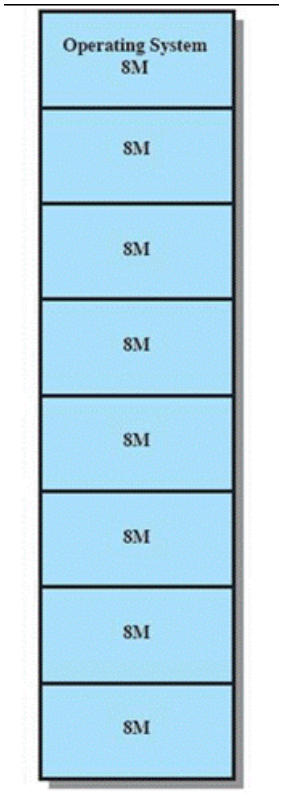
Contiguous memory management schemes:

- A process can be allocated with a free memory partition or block based three strategies:

1. First fit:
   - This type will allocate the first free block that is big enough to satisfy the memory request of a process from the beginning of the memory.
2. Best fit:
   - This type will allocate the smallest free block that is big enough to satisfy the memory request of a process by searching the whole memory.
3. Worst fit:
   - This will allocates the process to the partition, which is the largest sufficient freely available partition in the main memory.

Fixed Partitioning

- Static partitioning of physical memory
  - Main memory divided into a number of static equal-sized partitions
  - A process may be loaded into a partition of equal or greater size
- When all partitions are occupied, operating system may swap processes in and out of partitions
- Strength:
  - Simple to implement
- Weakness:
  - May leads to ***internal fragmentation***



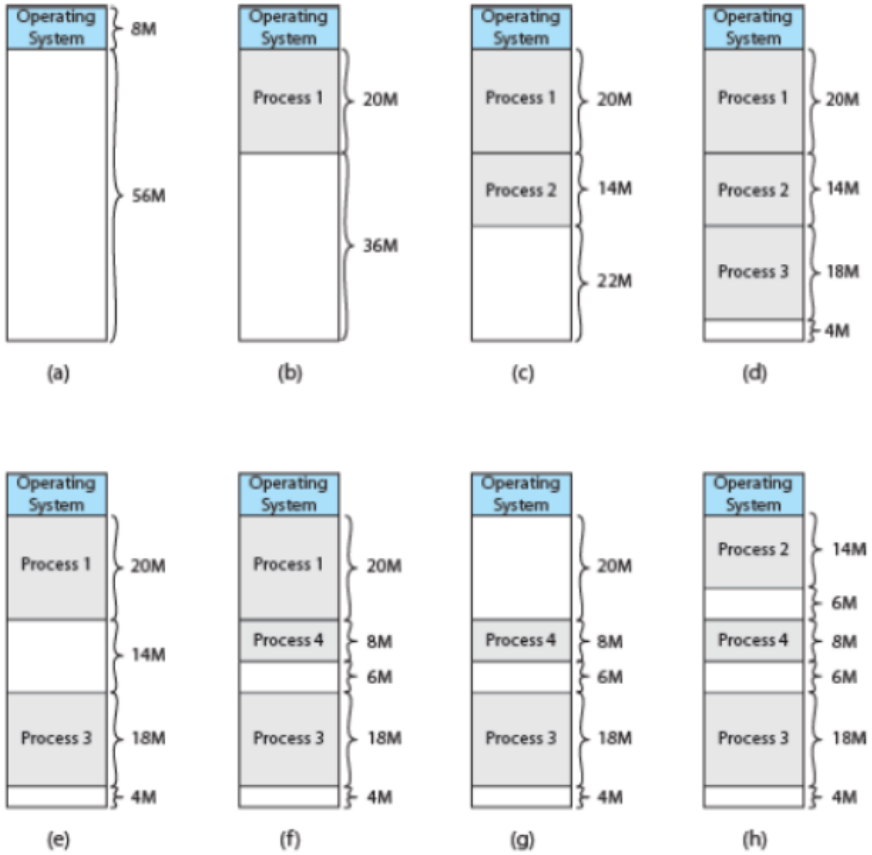
Internal Fragmentation

- Programs may not fit into a partition
- Inefficient use of main memory
  - A program will occupy a complete partition, even if it is small and does not need the complete memory allocated
  - Waste of memory: a partition is internally fragmented, free memory fragments of a partition cannot be given to other processes
- This phenomenon is called internal fragmentation
  - the block of data loaded (swapped in) is smaller than the partition

Dynamic Partitioning

- Partitions are of variable length and number
- Processes that are swapped in, are allocated the exact amount of memory needed
- This leads to ***external fragmentation*** of memory
  - At the beginning there is one contiguous memory area
  - The swapping in/out of processes and allocation of memory area will lead to many small un-allocated memory between the parts assigned to processes

Effect of Dynamic Partitioning



External Fragmentation

- The memory “holes” between assigned partitions may be too small for any process to fit in – waste of memory
- This phenomenon is called external fragmentation
  - Memory external to partitions becomes increasingly fragmented
  - Memory utilization declines

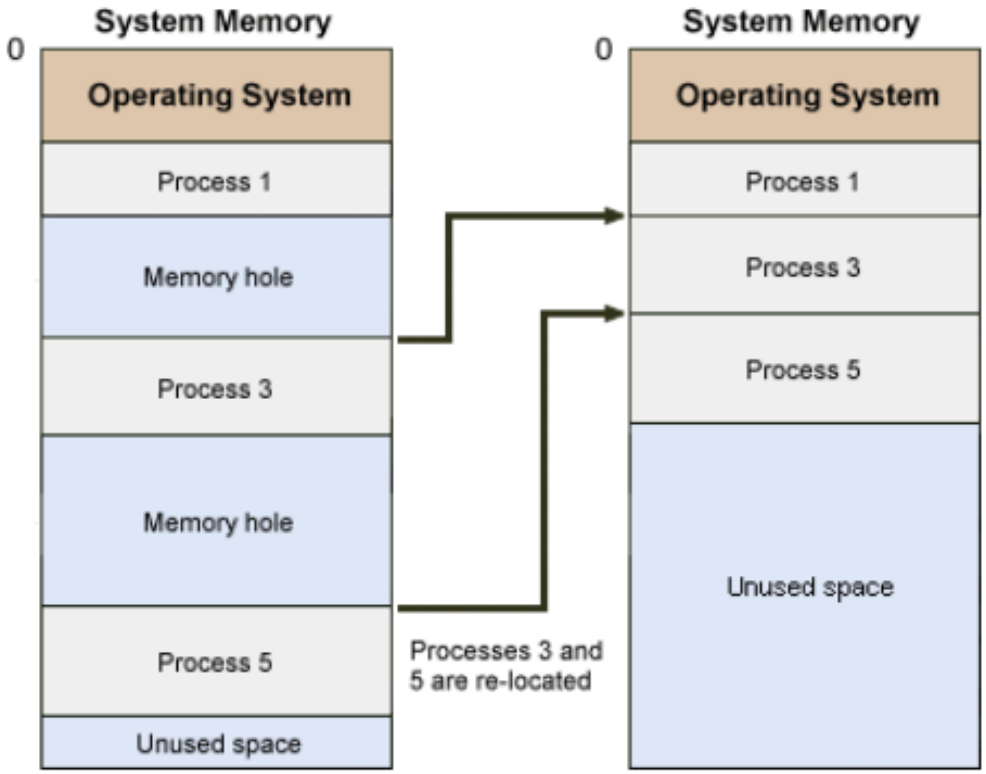
Compaction

- Compaction is a technique of collecting all the free spaces together in one block, so that other process can use this block or partition.

- There are large no. of small chunks of free memory that may be scattered all over the physical memory and individual each of chunks may not big enough to accommodate even a small program.

- So, compaction is a technique by which the small chunk of free spaces are made contiguous to each other into a single free partition, that may be big enough to accommodate some other processes

Compaction


System Memory
0
Operating System
Process 1
Memory hole
Process 3
Memory hole
Process 5
Unused space
Processes 3 and 5 are re-located
System Memory
0
Operating System
Process 1
Process 3
Process 5
Unused space

Non-Contiguous Memory Allocation

- The Non-contiguous memory allocation allows a process to acquire the several memory blocks at the different location in the memory according to its requirement.
- The non-contiguous memory allocation also reduces the memory wastage caused due to internal and external fragmentation.
- As it utilizes the memory holes, created during internal and external fragmentation.
  - The available free memory space are scattered here and there and all the free memory space is not at one place. So this is time-consuming.
  - A process will acquire the memory space but it is not at one place it is at the different locations according to the process requirement.
  - It reduces the wastage of memory which leads to internal and external fragmentation. This utilizes all the free memory space which is created by a different process.

Non-Contiguous Memory Allocation

- Non-contiguous memory allocation is of different types,

1. Paging
2. Segmentation
3. Segmentation with paging

Paging

- Paging is another memory-management scheme that also permits non-contiguous physical address space.
- In this the main memory is divided into equal size partitions.
- The partitions of the secondary memory are called pages while the partitions of the main memory are called frames.
- They are divided into equal size partitions to have maximum utilization of the main memory and hence avoid external fragmentation.

➢ **Translation of logical Address into physical Address**

- The logical address consists of two parts page number and page offset.

1. ***Page Number:*** It tells the exact page of the process which the CPU wants to access.
2. ***Page Offset***: It tells the exact word on that page which the CPU wants to read.

Paging

- Physical Address: The physical address consists of two parts frame number and page offset.

1. ***Frame Number:*** It tells the exact frame where the page is stored in physical memory.
2. ***Page Offset:*** It tells the exact word on that page which the CPU wants to read. It requires no translation as the page size is the same as the frame size so the place of the word which CPU wants access will not change.

- **Page table:** A page table contains the frame number corresponding to the page number of some specific process. So, each process will have its own page table. A register called Page Table Base Register (PTBR) which holds the base value of the page table.

Paging

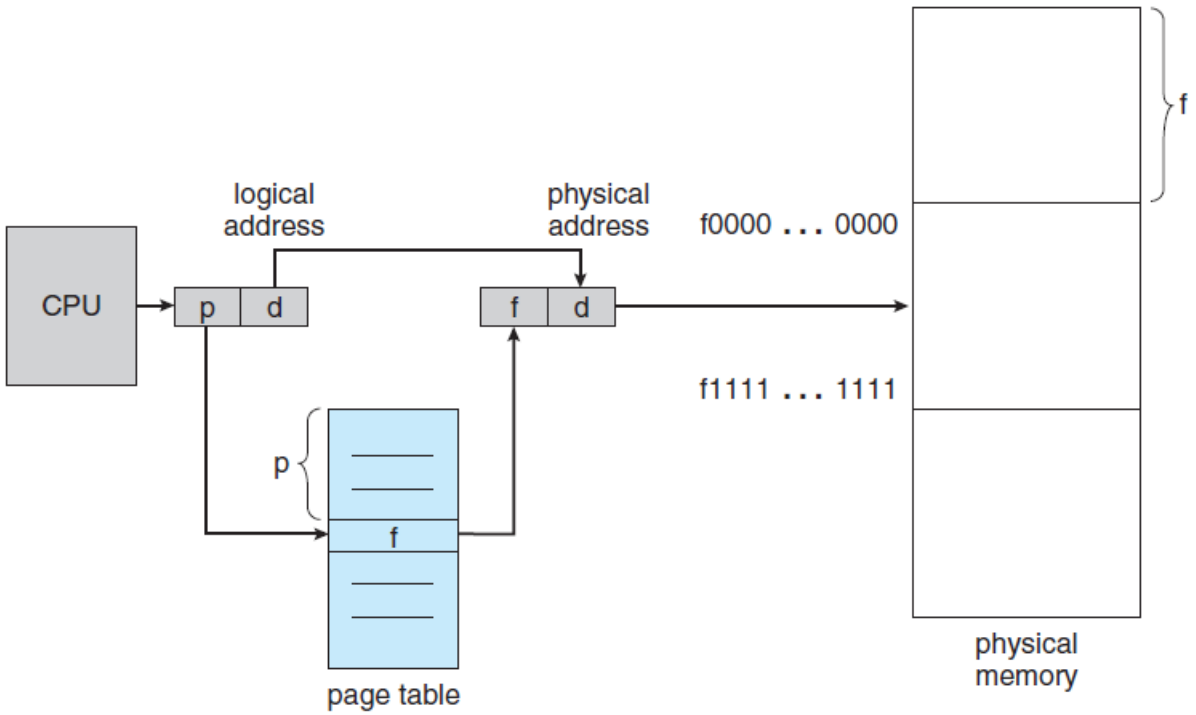


Figure 8.10 Paging hardware.

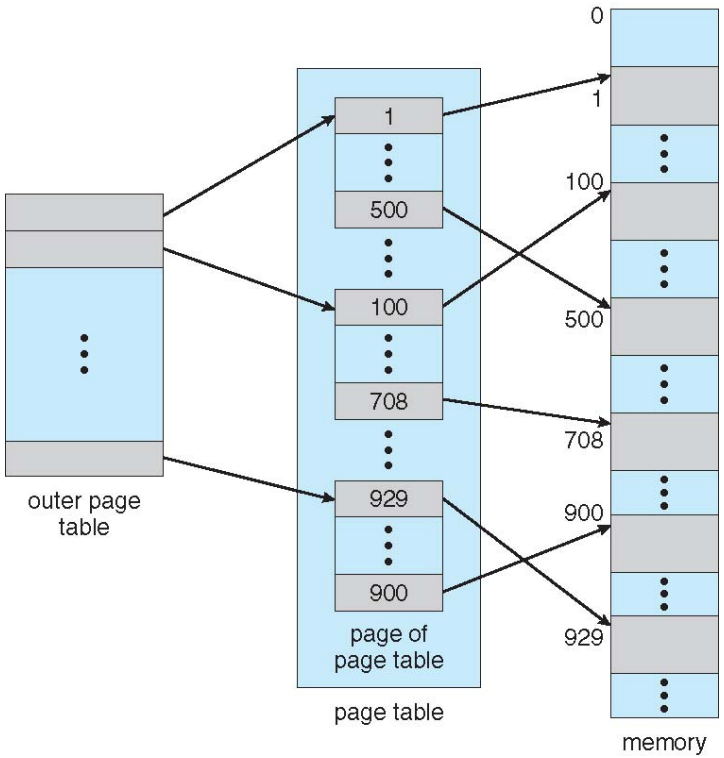
Structure of the Page Table

1. Hierarchical Paging
2. Hashed Page Tables
3. Inverted Page Tables

Hierarchical Paging

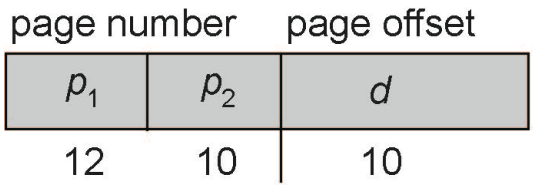
- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table
- We then page the page table

Two-Level Page-Table Scheme



Two-Level Paging Example

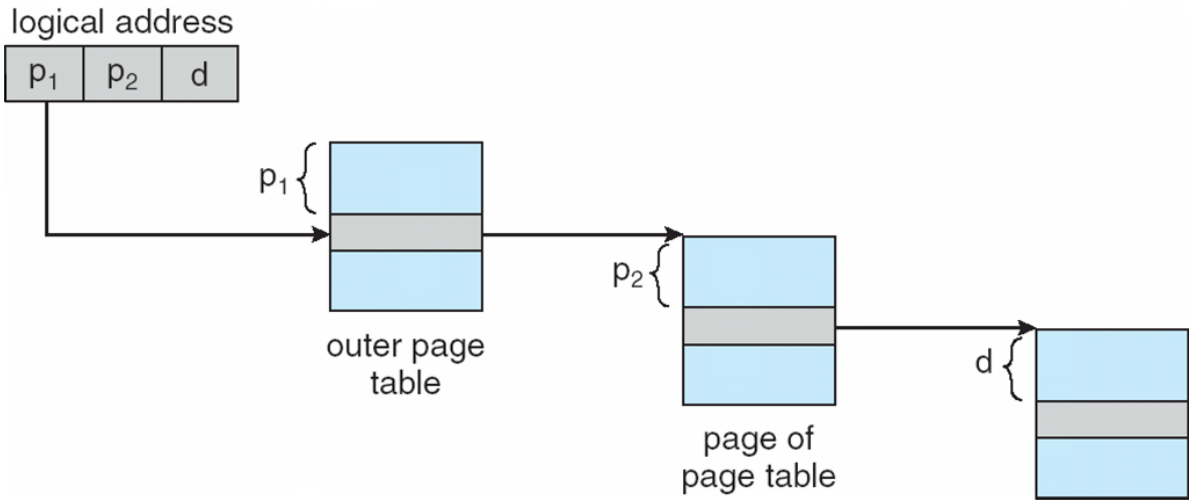
- A logical address (on 32-bit machine with 1K page size) is divided into:
  - a page number consisting of 22 bits
  - a page offset consisting of 10 bits
- Since the page table is paged, the page number is further divided into:
  - a 12-bit page number
  - a 10-bit page offset
- Thus, a logical address is as follows:



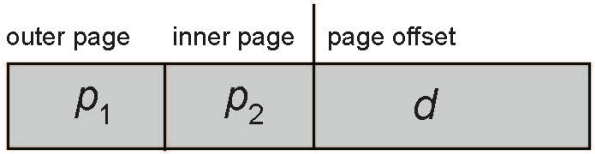
where *p1* is an index into the outer page table, and *p2* is the displacement within the page of the inner page table

- Known as **forward-mapped page table**

Address-Translation Scheme



64-bit Logical Address Space

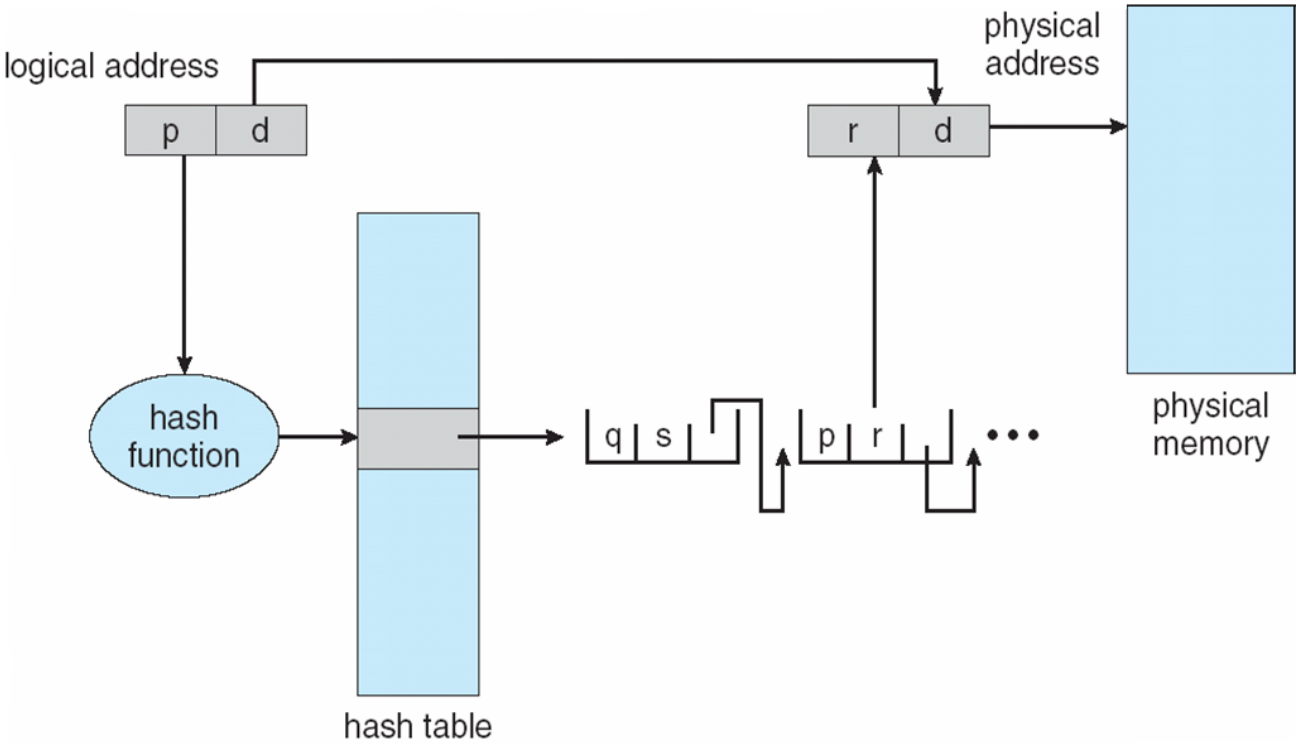
- Even two-level paging scheme not sufficient
- If page size is 4 KB (212)
  - Then page table has 252 entries
  - If tw 

tables could be 210
4-byt
  - Addr
  - Outer page table has 242 entries or 244 bytes

Hashed Page Tables

- Common in address spaces > 32 bits
- The virtual page number is hashed into a page table
  - This page table contains a chain of elements hashing to the same location
- Each element contains (1) the virtual page number (2) the value of the mapped page frame (3) a pointer to the next element
- Virtual page numbers are compared in this chain searching for a match
  - If a match is found, the corresponding physical frame is extracted
- Variation for 64-bit addresses is **clustered page tables**
  - Similar to hashed but each entry refers to several pages (such as 16) rather than 1
  - Especially useful for **sparse** address spaces (where memory references are non-contiguous and scattered)

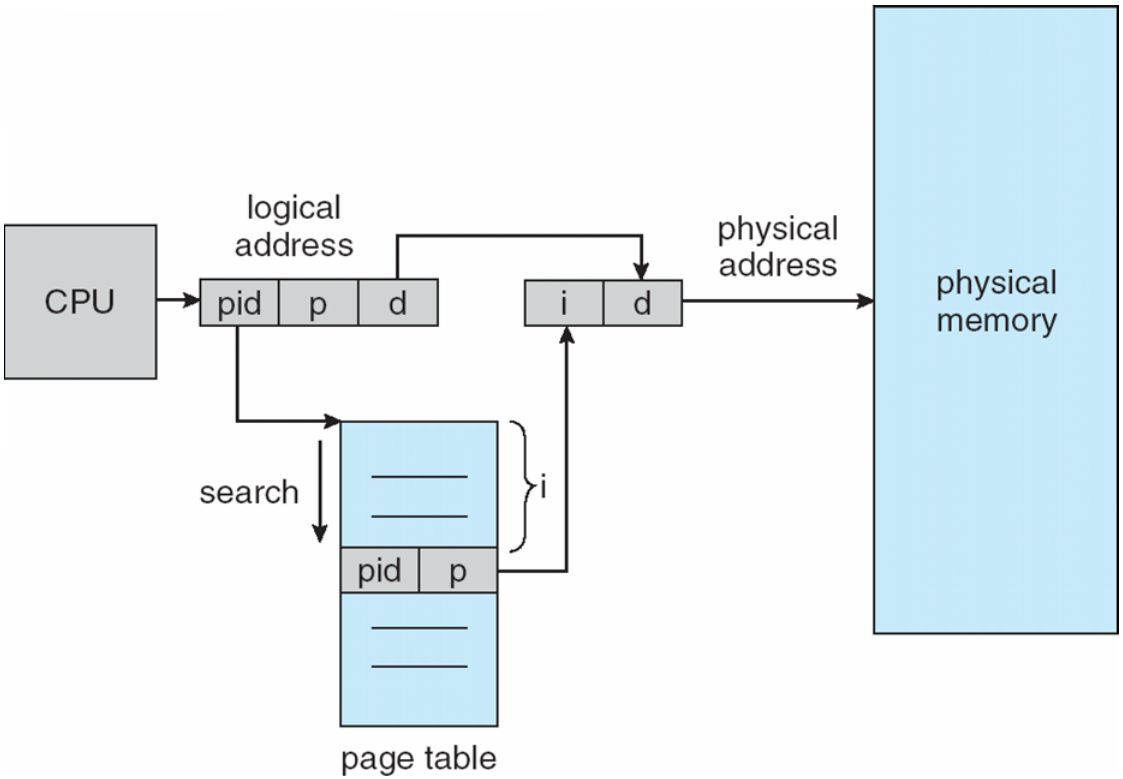
Hashed Page Tables



Inverted Page Table

- Rather than each process having a page table and keeping track of all possible logical pages, track all physical pages
- One entry for each real page of memory
- Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page
- Decreases memory needed to store each page table, but increases time needed to search the table when a page reference occurs
- Use hash table to limit the search to one — or at most a few — page-table entries
  - TLB can accelerate access
- But how to implement shared memory?
  - One mapping of a virtual address to the shared physical address

Inverted Page Table



Paging

- ***Advantages of Paging***

1. There is no external fragmentation as it allows us to store the data in a non-contiguous way.
2. Swapping is easy between equal-sized pages and frames.

- ***Disadvantages of Paging***

3. As the size of the frame is fixed, so it may suffer from internal fragmentation.
4. The access time increases because of paging as the main memory has to be now accessed two times.
5. For every process, we have an independent page table and maintaining the page table is extra overhead.

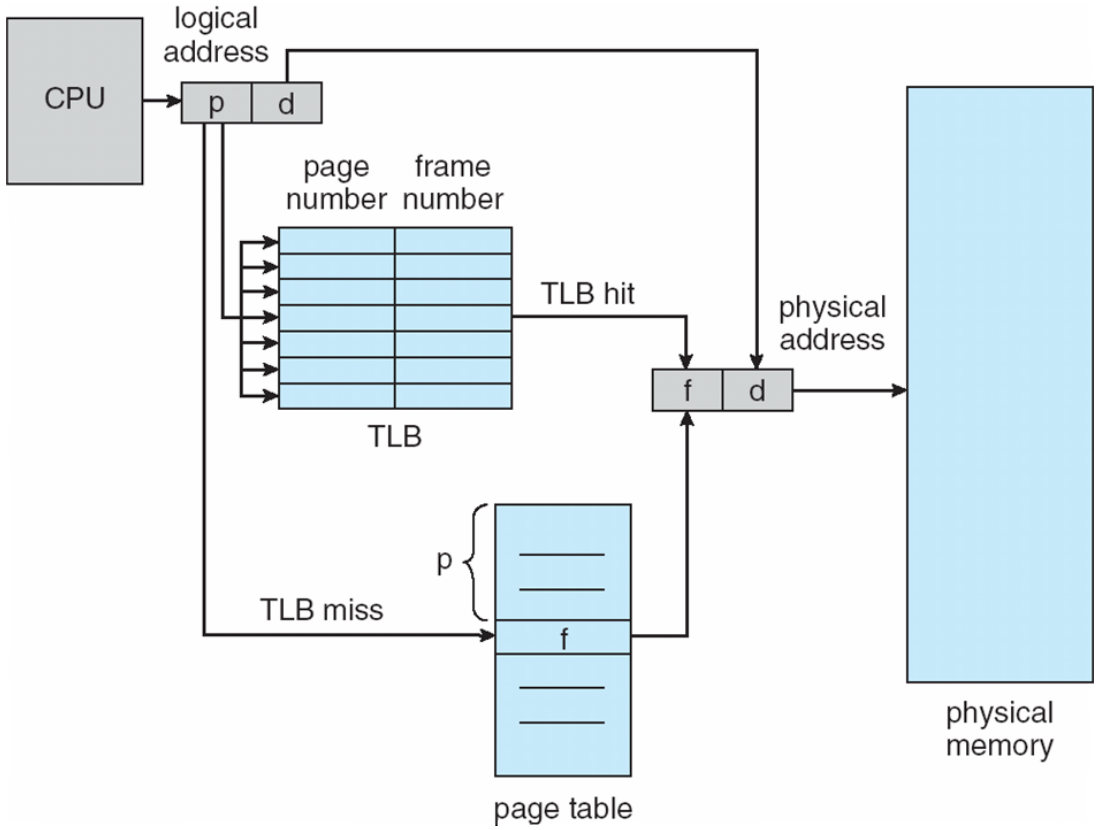
Paging with Translation Look-aside Buffers (TLBs)

- Since Page table is kept in main memory it can be accessed by the pointer Page-table base register (PTBR)
- Page-table length register (PTLR) indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
- One for the page table and one for the data / instruction
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called associative memory or translation look-aside buffers (TLBs)
- Each TLB entry – uniquely identifies each process to provide address-space protection for that process.
- TLBs are typically small size compared to Page Table.

Paging with Translation Look-aside Buffers (TLBs)

- Each time when address translation need to be done, the page reference will be checked in the TLB first.
- If it present then it is called TLB Hit, then its frame number will be passed. If it is not present in the table then it called TLB Miss.
- On a TLB miss, then page will be searched in the Page Table and later value is loaded into the TLB for faster access next time.
- Paging hardware with TLB consists of both TLB table and Page Table.

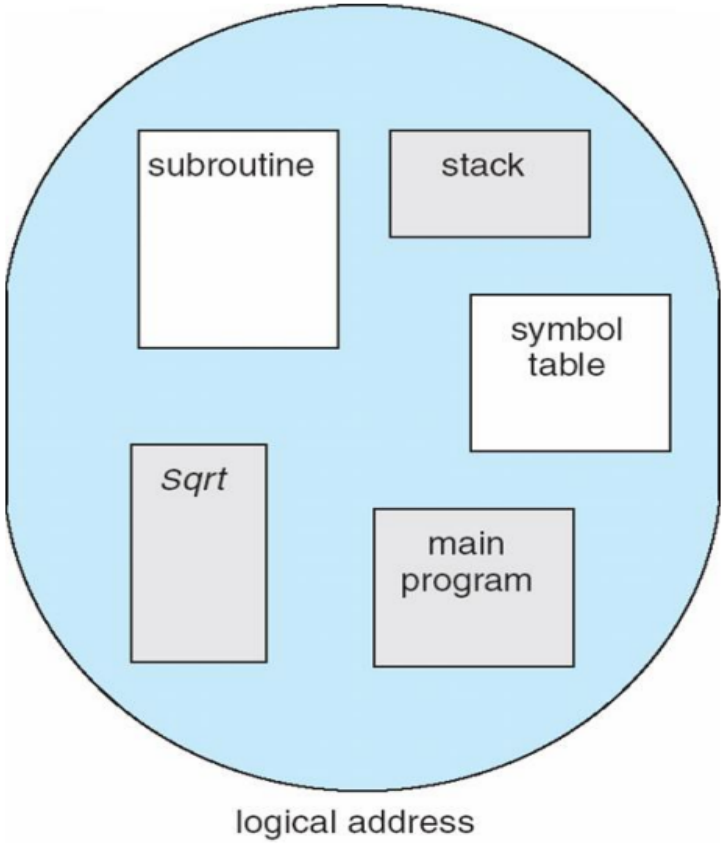
Paging with Translation Look-aside Buffers (TLBs)



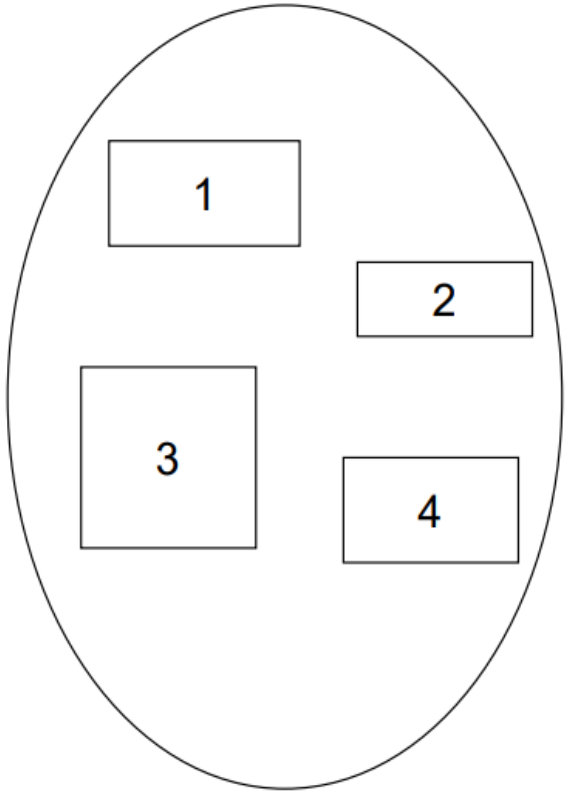
Segmentation

- Segmentation is a memory-management approach that is implemented by segmenting processes and loading them into different non-contiguous addressed spaces in memory with different segment address.
- A program is a collection of segments
  - A segment is a logical unit such as:
    - main program
    - procedure
    - function
    - method
    - object
    - local variables, global variables
    - common block
    - stack
    - symbol table
    - arrays

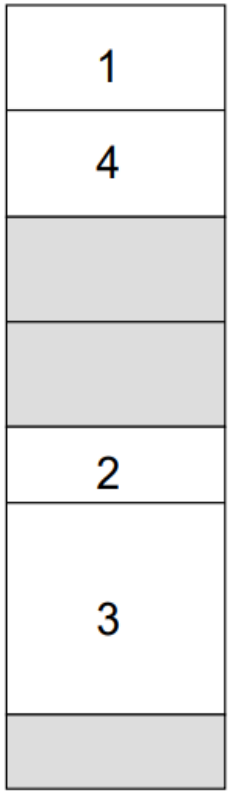
User's View of a Program



Logical View of Segmentation



user space



physical memory space

Segmentation Architecture

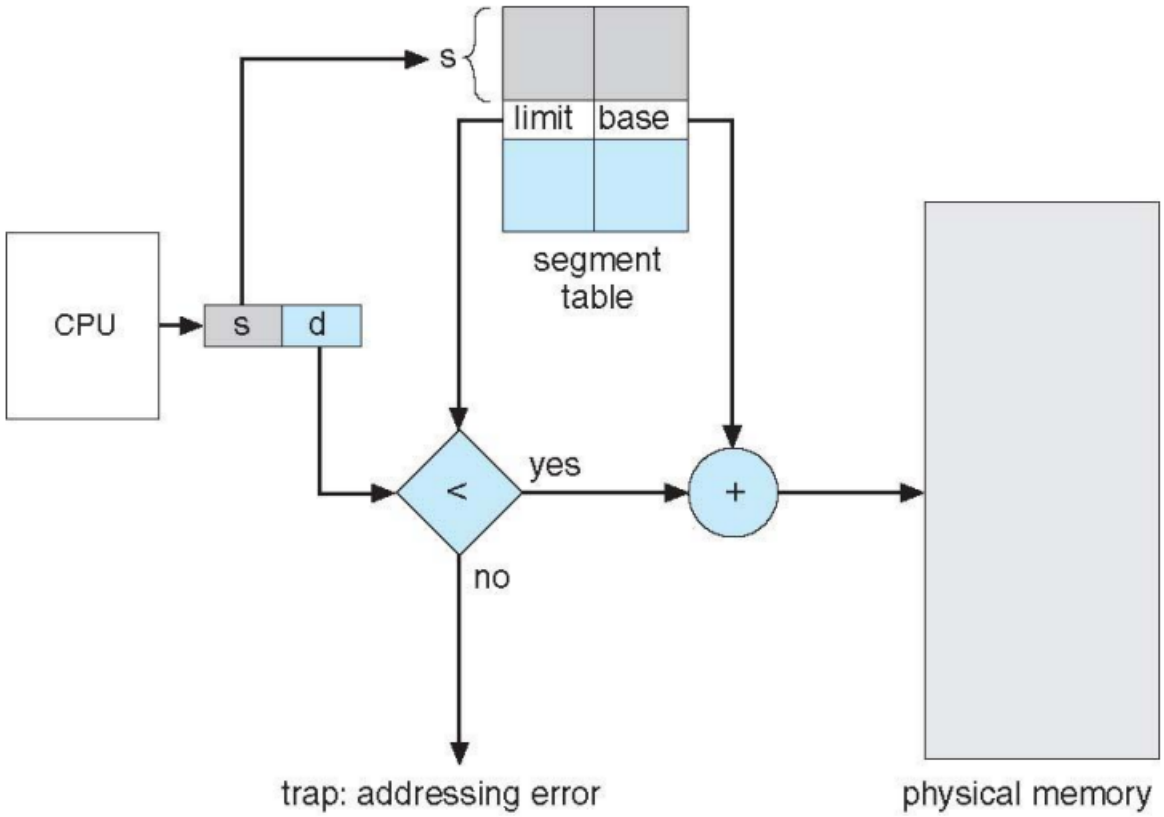
- Logical address consists of a two tuple:
  < Segment_number, Offset >
- ***Segment table:–*** maps two-dimensional physical addresses; each table entry has:
- ***Base:–*** contains the starting physical address where the segments reside in memory
- ***Limit:–*** specifies the length of the segment
- Segment-table base register **(STBR)** points to the segment table's location in memory
- Segment-table length register **(STLR)** indicates number of segments used by a program;
  segment number s is legal if s < STLR

Segmentation Architecture

- Protection
  - With each entry in segment table associate:
    - validation bit = 0 => illegal segment
    - read/write/execute privileges

- Protection bits associated with segments; code sharing occurs at segment level
- Since segments vary in length, memory allocation is a dynamic storage-allocation problem
- A segmentation example is shown in the following diagram

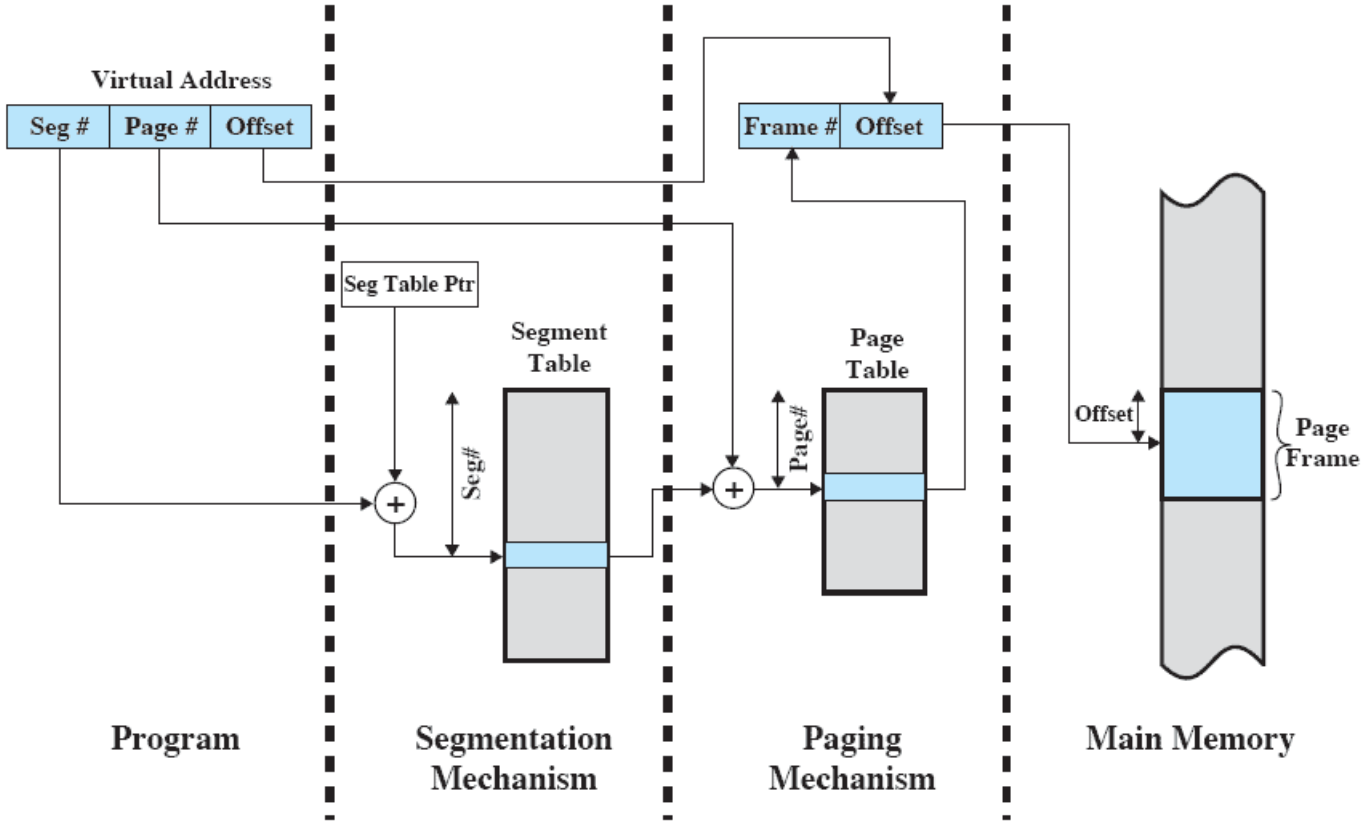
Segmentation Hardware



Segmentation and Paging

- In segmentation with paging, we take advantages of both segmentation as well as paging.
- It is a kind of multilevel paging but in multilevel paging, we divide a page table into equal size partition but here in segmentation with paging, we divide it according to segments.
- All the properties are the same as that of paging because segments are divided into pages.

Segmentation and Paging



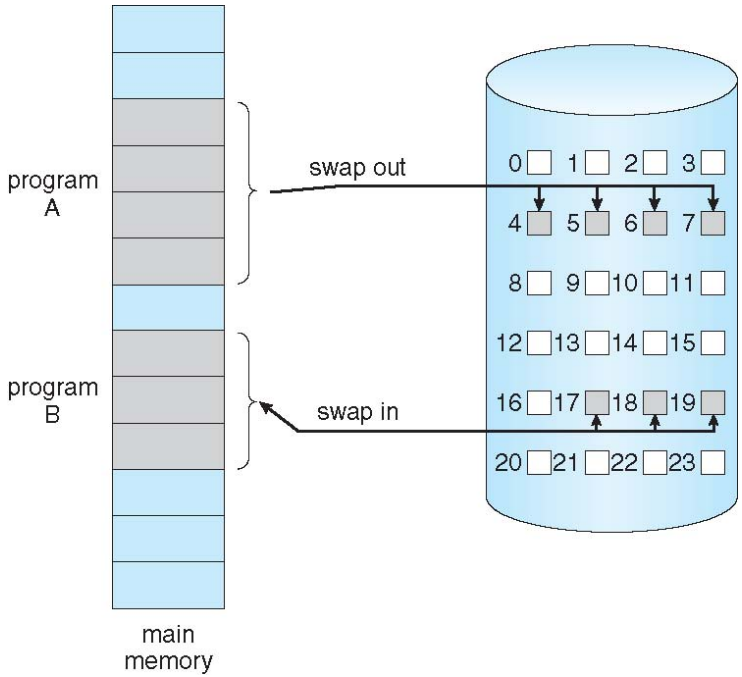
Virtual Memory

- Virtual Memory is a storage mechanism which offers user an illusion of having a very big main memory.
- It is done by treating a part of secondary memory as the main memory.
- In Virtual memory, the user can store processes with a bigger size than the available main memory.
- Instead of loading one long process in the main memory, the OS loads the various parts of more than one process in the main memory.
- Virtual memory is mostly implemented with demand paging and demand segmentation.

Demand Paging

- Demand paging is a memory management technique used by operating systems to optimize the use of memory resources.
- In demand paging, we only load the required pages of a process into the main memory instead of loading the entire process.
- A demand paging mechanism is very much similar to a paging system with swapping where processes stored in the secondary memory and pages are loaded only on demand, not in advance.
- During the program execution, if the program references a page that may not be available in the main memory then this scheme will bring that page on demand from the secondary storage.

Demand Paging



- It uses a Lazy swapper – never swaps a page into memory unless page will be needed.
- Swapper that deals with pages is called Pager.

Page Fault

If there is a reference to a page, first reference to that page will trap to operating system:

Steps to follow during page fault:

1.Operating system looks at another table to decide:

Invalid reference abort

Just not in memory

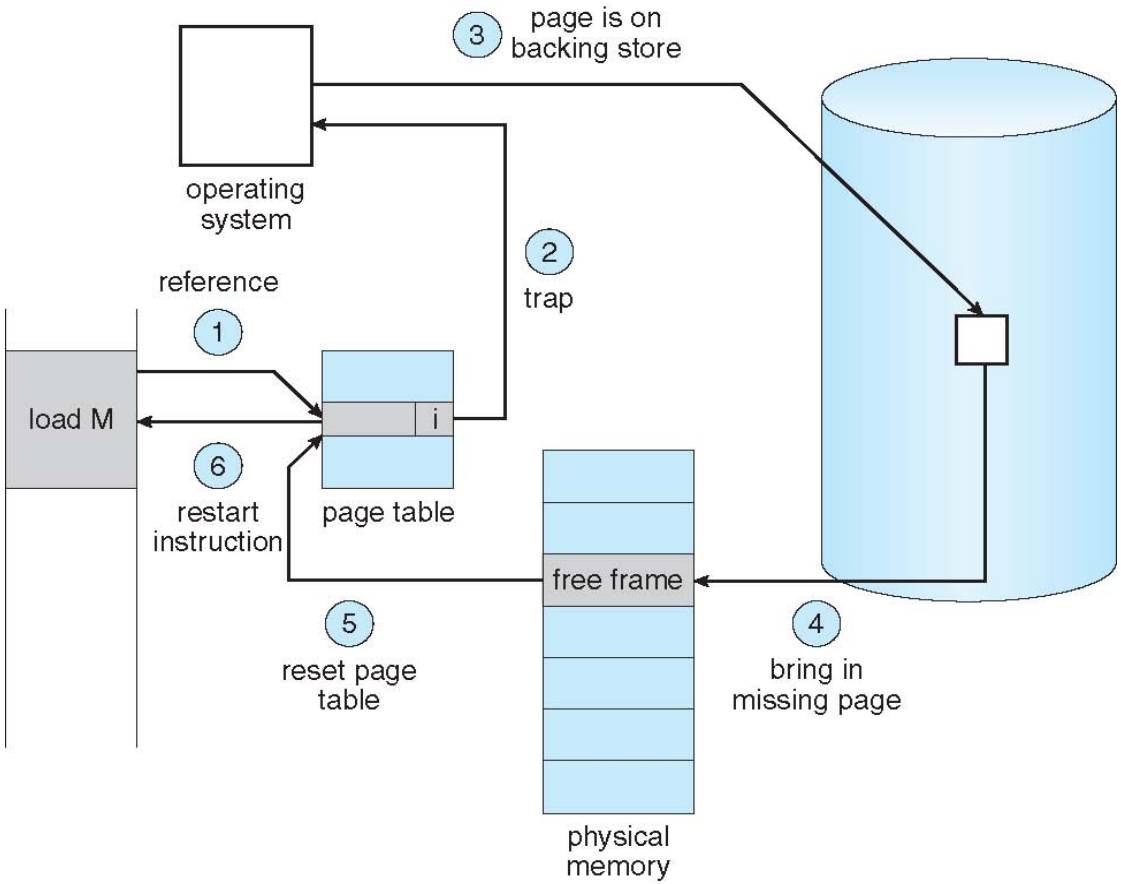
2.Find free frame

3.Swap page into frame via scheduled disk operation

4.Reset tables to indicate page now in memory
Set validation bit = v

5.Restart the instruction that caused the page fault

Page Fault


3
page is on
backing store
operating
system
2
trap
reference
1
load M
i
6
restart
instruction
page table
free frame
5
reset page
table
4
bring in
missing page
physical
memory

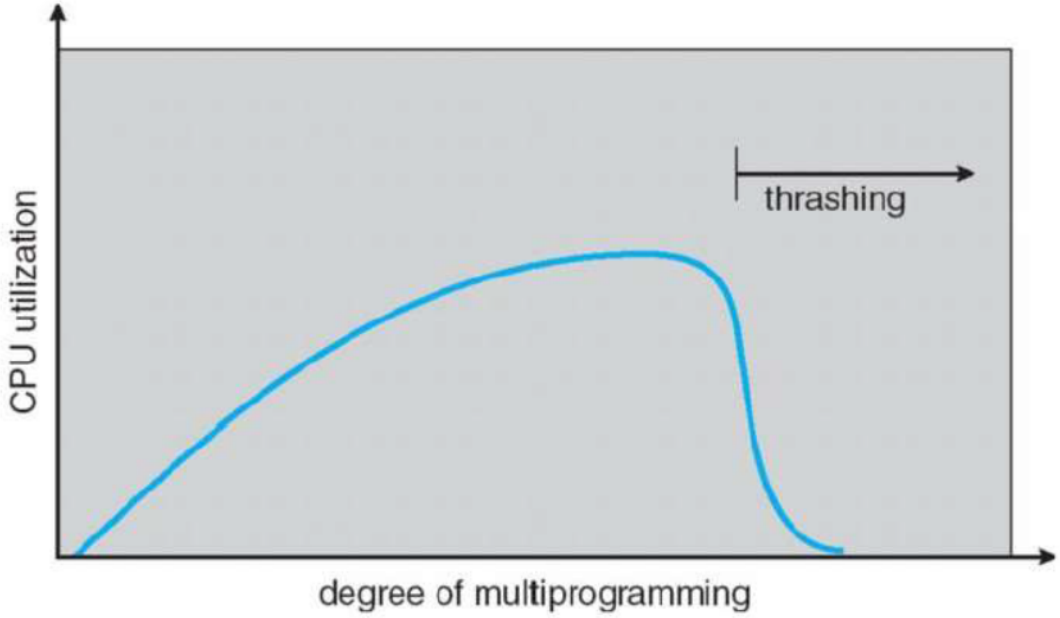
Thrashing

Thrashing is a condition or a situation where the operating system is spending a major portion of its time in servicing the page faults, rather than actual processing.

The effect of Thrashing is:

- CPU becomes idle.
- Decreasing the utilization increases the degree of multiprogramming and hence bringing more processes at a time which in fact increases the thrashing exponentially.

Thrashing



Page Replacement Algorithm

- Page Replacement Algorithm decides which page to remove, also called swap out when a new page needs to be loaded into the main memory.
- Page Replacement happens when a requested page is not present in the main memory and the available space is not sufficient for allocation to the requested page.
- A page replacement algorithm tries to select which pages should be replaced so as to minimize the total number of page misses.
- If a process requests for page and that page is found in the main memory then it is called page hit.
- If the requested page is not found in the main memory then it is called as page miss or page fault.

Page Replacement Algorithm

- Some Page Replacement Algorithms :

1. First In First Out (FIFO) Algorithm
2. Least Recently Used (LRU) Algorithm
3. Optimal Page Replacement Algorithm
4. Second Chance Page Replacement Algorithm
5. Working Set Page Replacement Algorithm
6. WS Clock Page Replacement Algorithm

First In First Out (FIFO)

- This is the simplest page replacement algorithm. In this algorithm, the OS maintains a queue that keeps track of all the pages in memory, with the oldest page at the front and the most recent page at the back.

- When there is a need for page replacement, the FIFO algorithm, swaps out the page at the front of the queue, that is the page which has been in the memory for the longest time.

Advantages

- Simple and easy to implement.
- Low overhead.

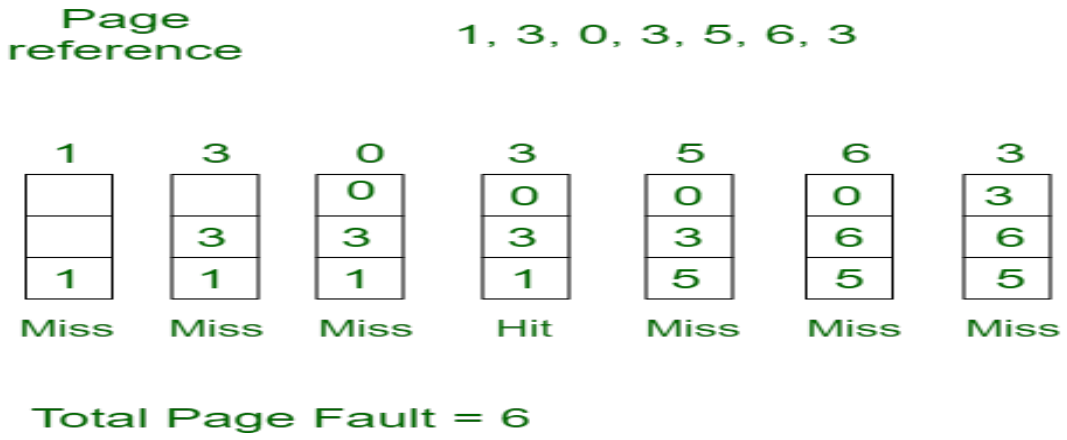
Disadvantages

- Poor performance.
- Suffers from Belady's Anomaly(i.e. more page faults when we increase the number of page frames).

First In First Out (FIFO)

For Example:

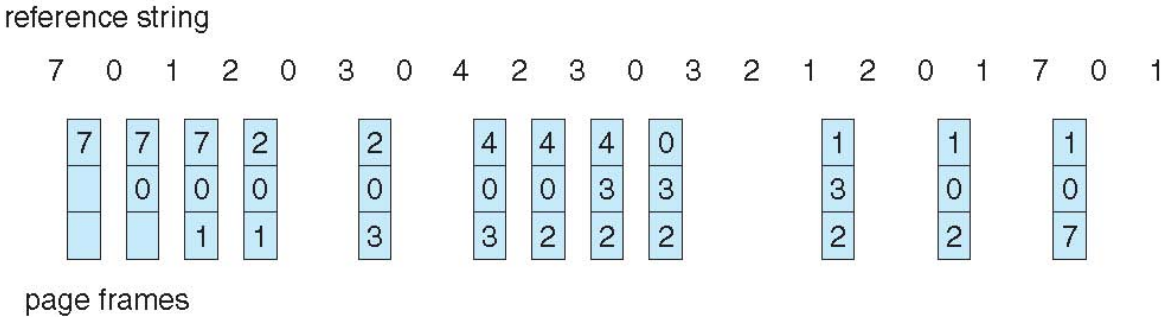
Consider page reference string 1, 3, 0, 3, 5, 6 with 3 page frames. Find number of page faults.



Belady's anomaly – Belady's anomaly proves that it is possible to have more page faults when increasing the number of page frames while using the First in First Out (FIFO) page replacement algorithm.

Least Recently Used (LRU) Algorithm

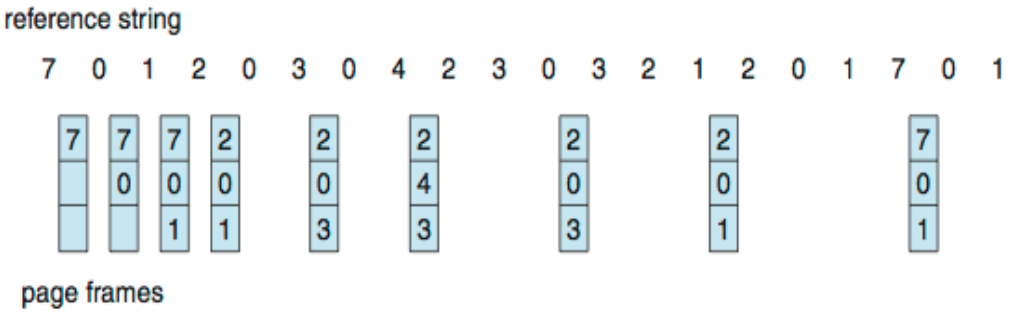
- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page.



- 12 faults – better than FIFO but worse than OPT

Optimal Algorithm

- Replace page that will not be used for longest period of time
- OS needs future knowledge of page references in advance.
- Need to sort according to their next usage/ reference
- Overhead for OS & complex to implement.



Second Chance Algorithm

- The Second Chance Page Replacement Algorithm, is an enhanced version of the FIFO page replacement algorithm.
- The main issue with the FIFO algorithm is that it may remove frequently used pages just because they were loaded earlier.
- To overcome this, the Second Chance algorithm gives pages a second chance before replacing them.
- This algorithm is also known as the ***Clock Page Replacement*** algorithm because of its circular structure.
- In this algorithm, each page has a reference bit associated with it.

➢ When a page is accessed, its reference bit is set to 1.

➢ When a page needs to be replaced, the algorithm checks the reference bit of the pages in the order they were loaded (like FIFO).

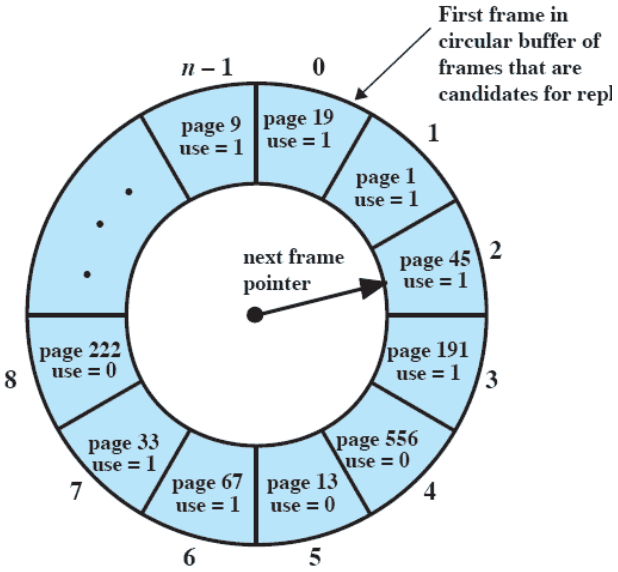
- If the reference bit is 0, the page is replaced.
- If the reference bit is 1, the page is given a second chance and then its reference bit is cleared (to ensure no third chance is given).

Steps of Second Chance Page Replacement Algorithm

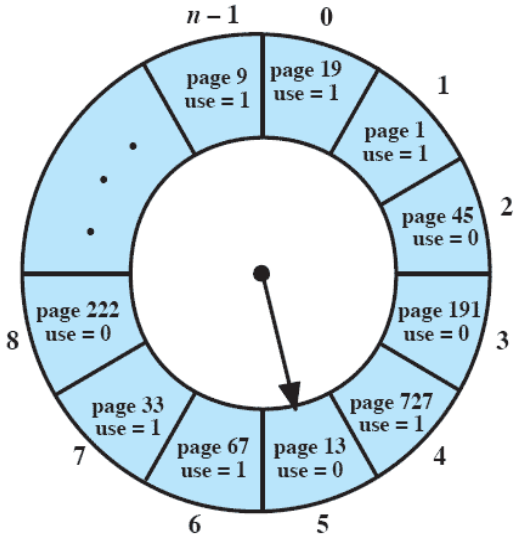
To implement the Second Chance Page Replacement Algorithm, follow these steps −

1. Maintain a circular queue of pages in memory, same as that of FIFO algorithm.
2. Create a pointer that points to the oldest page in the queue.
3. For each page, create a reference bit (R) initialized to 0. When a page is accessed, set its reference bit (R) to 1.
4. When a page fault occurs and a page needs to be replaced, check the page pointed to by the pointer. If its reference bit (R) is 0, replace that page with the new page.
5. If the reference bit (R) is 1, give the page a second chance by clearing its reference bit (set R to 0) and move the pointer to the next page in the queue.
6. Repeat this process until a page with reference bit 0 is found.

Second Chance Algorithm



(a) State of buffer just prior to a page replacement



(b) State of buffer just after the next page replacement

Working Set

- A working set is an approximation of a program's locality
- Uses the most recent page references from a program to determine which pages are most important
  - Parameter Δ : the most recent Δ page references
- Defines a "working set window"
  - The set of pages that are referenced within the recent Δ page references
  - A page in active use is part of the working set window

Working Set

- Parameter Δ is an important tuning parameter
  - If Δ is too small, not all actively used pages will be in the working set window
  - If Δ is too large, it may overlap several localities
- Working set size *WSSi*
  - Size of the working set for process *i*
  - Process is actively using all the pages in its working set
  - Therefore, a process needs at least *WSSi* frames to avoid thrashing
  - Depends on Δ

Working Set

- How many processes can be started?
  - Operating system will start processes and allocate frames as needed by their working sets
  - Working sets can change in size and need for frames
  - If the sum of all needed frames exceeds the actually available number of frames, one of the processes is selected for suspension
    - Its pages are swapped out and its frames reallocated to other processes
  - The suspended process can be restarted later, when enough frames become available again

WS-Clock Page Replacement

- All pages are kept in a circular list.
- As pages are added, they go into the ring.
- The “clock hand” advances around the ring.
- Each entry contains “time of last use”.
- Upon a page fault...
  - If Reference Bit = 1...
    - Page is in use now. Do not evict.
    - Clear the Referenced Bit.
    - Update the “time of last use” field.

WS-Clock Page Replacement

- If Reference Bit = 0
  - If the age of the page is less than T...
    - This page is in the working set.
    - Advance the hand and keep looking
  - If the age of the page is greater than T...
    - If page is clean
      - Reclaim the frame and we are done!
    - If page is dirty
      - Schedule a write for the page
      - Advance the hand and keep looking

S.NO.	Contiguous Memory Allocation	Non-contiguous Memory Allocation
1	Contiguous memory allocation allocates consecutive blocks of memory to a file/process.	Non-Contiguous memory allocation allocates separate blocks of memory to a file/process.
2	Faster in Execution.	Slower in Execution.
3	It is easier for the OS to control.	It is difficult for the OS to control.
4	Overhead is minimum as not much address translations are there while executing a process.	More Overheads are there as there are more address translations.
5	Internal fragmentation occurs in Contiguous memory allocation method.	External fragmentation occurs in Non-Contiguous memory allocation method.

6	It includes single partition allocation and multi-partition allocation.	It includes paging and segmentation.
7	Wastage of memory is there.	No memory wastage is there.
8	In contiguous memory allocation, swapped-in processes are arranged in the originally allocated space.	In non-contiguous memory allocation, swapped-in processes can be arranged in any place in the memory.

S.NO	Internal Fragmentation	External Fragmentation
1	It occurs when free space within a memory block becomes unused.	It occurs when non contiguous free blocks becomes unused due to dynamic allocation.
2	It happens when the method or process is larger than the memory.	It happens when the method or process is removed from memory.
3	The solution of internal fragmentation is best-fit block.	Solution of external fragmentation is compaction, paging and segmentation.
4	Internal fragmentation occurs when memory is divided into fixed sized partitions.	External fragmentation occurs when memory is divided into variable size partitions based on the size of processes.